

KOHLER AI BATHROOM DESIGNER

Prompts, AI Instructions & Workflow Documentation

[GitHub Repository Submission Reference](#)

SECTION 1 — PROJECT OVERVIEW

Purpose: The KOHLER AI Bathroom Designer is an AI-assisted interior planning system. It bridges the gap between user intent and actionable design by extracting room constraints, validating them against real-world logic, and deterministically matching them with the KOHLER product catalog to recommend a physically viable, budget-conscious bathroom bundle.

Input Paths: The system accepts two distinct input methods:

- **A. Natural-Language Description:** Users provide a free-text description of their bathroom (dimensions, budget, preferred style, and desired products).
- **B. USD/USDZ Architecture File:** Users upload a 3D spatial file containing physical room geometry and dimensions.

Output Generation: Both input paths converge into a standardized requirement format. These requirements are validated and passed into a deterministic recommendation engine. The final output includes curated product bundles (with compatibility scoring), pricing details, and a 2D conceptual SVG layout.

SECTION 2 — AI / LLM ARCHITECTURE

The AI component of this project is strictly bounded. The LLM is used **exclusively for natural language understanding** (intent extraction). It never selects products, invents prices, or reasons about spatial placement.

Configuration	Implementation Detail
LLM Provider	Anthropic (configured via LLM_PROVIDER, defaults to "anthropic").
Model	claude-haiku-4-5-20251001 (Overridable via LLM_MODEL). Chosen for fast, inexpensive structured extraction.
API Authentication	LLM_API_KEY loaded via python-dotenv.
Purpose of LLM	To extract unstructured text into a strict JSON schema of room dimensions, budget, theme, required categories, and preferences.
Input Format	System prompt + User's free-text string (via Messages API).
Output Format	Strict JSON object (max 512 tokens).

Demo Mode Behavior

The project implements a deterministic DEMO_MODE to allow reviewers to test the pipeline without an active API key.

- **Trigger:** Activated when the environment variable DEMO_MODE=true.
- **Mechanism:** It bypasses the API entirely. Routes text to parse_demo_requirements(), using

SECTION 3 — SYSTEM PROMPTS / LLM PROMPTS

The system utilizes a single, highly constrained prompt to extract requirements.

Prompt 1: Requirement-Extraction Assistant

- **Location:** `src/llm_parser.py` (Variable: `SYSTEM_PROMPT`)
- **Purpose:** Extract dimensions, budget, categories, and preferences into JSON without hallucinating products or prices.
- **Expected Input/Output:** User text → Strict JSON object.

Actual Prompt Source

You are a requirement-extraction assistant for a bathroom renovation planning tool. Your ONLY job is to read a customer's free-text description of the bathroom they want, and extract structured REQUIREMENTS from it. You are not a product catalog and you are not a designer. STRICT RULES: 1. Extract requirements only. Do not recommend, mention, or invent any specific product, brand, SKU, price, MRP, or dimension of a product. 2. Do not use, reference, or assume anything about a product catalog. You have no access to one and must not pretend otherwise. 3. Return ONLY the structured JSON described below. No prose, no explanation, no markdown code fences, no extra keys. 4. If a piece of information is not clearly stated or clearly implied by the customer's text, its value MUST be null. NEVER guess or invent a value that was not stated. 5. `required_categories` must contain ONLY values from this exact set: ["Toilet", "Smart Toilet", "Faucet", "Shower", "Vanity", "washbasin"]. Normalize natural language into these categories. 6. Room dimensions must be expressed in FEET as plain numbers (e.g. 8, 6.5). If the customer gives dimensions in another unit, convert to feet. 7. Budget must be normalized to a single plain numeric value in INR (Indian Rupees), with no currency symbol, no commas, no words. Understand Indian numbering expressions such as "2 lakh" -> 200000. 8. `theme` should be a short style word/phrase taken from what the customer said (e.g. "Modern", "Classic Luxury"). If no style/theme is mentioned, theme must be null. Do not invent a theme. Return EXACTLY this JSON structure and nothing else: { "length_ft": number or null, "width_ft": number or null, "budget_inr": number or null, "theme": string or null, "required_categories": [], "preferences": { "color": string or null, "water_efficiency_keyword": string or null, "feature_keywords": [] } }

SECTION 4 — REQUIREMENT EXTRACTION WORKFLOW

The NLP pipeline translates raw intent into structured constraints using the following flow (implemented in `app.py` and `llm_parser.py`):

```
User natural-language description ↓ Requirement parser (Anthropic or Regex Demo)
```

Structured Requirement Fields

- `length_ft / width_ft`: Bathroom dimensions in feet.
- `budget_inr`: Target budget represented as a raw float/int in Indian Rupees.
- `theme`: Aesthetic preference string (e.g., "Modern", "Classic").
- `required_categories`: Array of strings matching the catalog schema (e.g., "Smart Toilet", "Vanity").
- `preferences`: Soft constraints including `color`, `water_efficiency_keyword`, and `feature_keywords`.

SECTION 5 — REQUIREMENT VALIDATION

Validation ensures the recommendation engine receives logically sound data, averting crashes and invalid physics (implemented in `src/validation.py`).

- **Field Validation** (`validate_requirements`): Enforces types and bounds. `length_ft` and `width_ft` must be strictly `> 0`. `budget_inr` must be `>= 0`. Unrecognized categories are safely dropped.
- **Completeness Check**: Validates that the engine has the minimum required data to run. It ensures `length_ft`, `width_ft`, `budget_inr`, and `required_categories` are not null/empty.
- **Error Handling**: Malformed inputs raise a `RequirementsValidationError`. Incomplete inputs return a safe `valid: False` flag, prompting the UI to request missing fields.
- **Adapter Translation**: Remaps keys (e.g., `budget_inr` to `budget`) to match engine kwargs.

SECTION 6 — USD / USDZ WORKFLOW

The architecture upload pipeline is strictly deterministic and operates independently of the LLM.

```
USD/USDZ File ↓ OpenUSD Parser (Regex/Attribute Extraction) ↓ Room dimensions +
```

Implementation Details (usd_parser.py & architecture_adapter.py)

- **No AI Interpretation:** The `parse_usd_file()` function traverses the OpenUSD stage programmatically to extract `roomLength`, `roomWidth`, and components.
- **Explicit Intent Required:** Detecting an existing "toilet" or "washbasin" in the file provides spatial context, but it *does not* automatically inject them as required products. The user must manually select their desired product categories in the UI.
- **Adapter:** `architecture_to_requirements()` merges the parsed room geometries with the user-selected categories and budget into the standardized requirements dictionary.

SECTION 7 — RECOMMENDATION WORKFLOW

Once requirements are standardized, they enter `recommendation_engine.py`. This phase is **100% deterministic Python logic**; the LLM is no longer involved.

- **Filtering:** Products are filtered by required categories. Products with known spatial conflicts (`check_product_space`) are eliminated. Note: Products with missing dimensions in the catalog are retained, preventing false negatives.
- **Bundle Generation:** Valid candidates are permuted into bundles (`generate_bundles`). Only bundles matching the total budget limit are kept.
- **Compatibility Scoring (`calculate_bundle_score`):** Bundles are ranked on a 0-100 scale using a weighted formula:
 - **40% Style Match:** Product style vs. requested theme.
 - **25% Budget Efficiency:** How effectively the budget is utilized.
 - **20% Spatial Fit:** Ratio of product footprint vs. room area.
 - **15% Feature Match:** Alignment with color and water efficiency keywords.

SECTION 8 — WHAT-IF WORKFLOW

The "What-If" redesign feature allows users to iterate on their recommended layout without restarting.

- **Interpretation:** The user submits a text change (e.g., "Make it more luxurious and add a shower"). This is parsed using the same LLM/Demo pipeline (`parse_requirements` or `parse_demo_changes`).
- **Delta Modification:** The `merge_what_if_requirements()` function in `app.py` applies these changes to the existing baseline. It explicitly handles additive (e.g., "add shower") and subtractive (e.g., "remove bathtub") language for categories while overwriting scalars like budget or theme.
- **Regeneration:** The merged requirement object is re-validated and fed back into the deterministic recommendation engine to generate new layout bundles.

SECTION 9 — AI SAFETY / VALIDATION / ERROR HANDLING

The application implements robust safeguards across the pipeline to prevent hallucinations and crashes:

Safety Mechanism	Location	Description
Prompt Boundaries	<code>llm_parser.py</code>	Strictly forbids hallucinating products, prices, or SKUs, limiting the AI to intent extraction.
Type/Value Enforcement	<code>validation.py</code>	Ensures dimensions and budgets are strictly positive numeric values. Prevents physics-breaking inputs.
Category Sanitization	<code>validation.py</code>	Unrecognized categories hallucinated by the LLM (e.g., "Jacuzzi") are silently dropped.
Exception Routing	<code>app.py</code>	Errors are typed (e.g., <code>LLMResponseValidationError</code>) and gracefully mapped to UI warnings.
Demo Mode Sandbox	<code>app.py</code>	Provides a safe fallback (<code>DEMO_MODE=true</code>) that avoids network calls entirely.

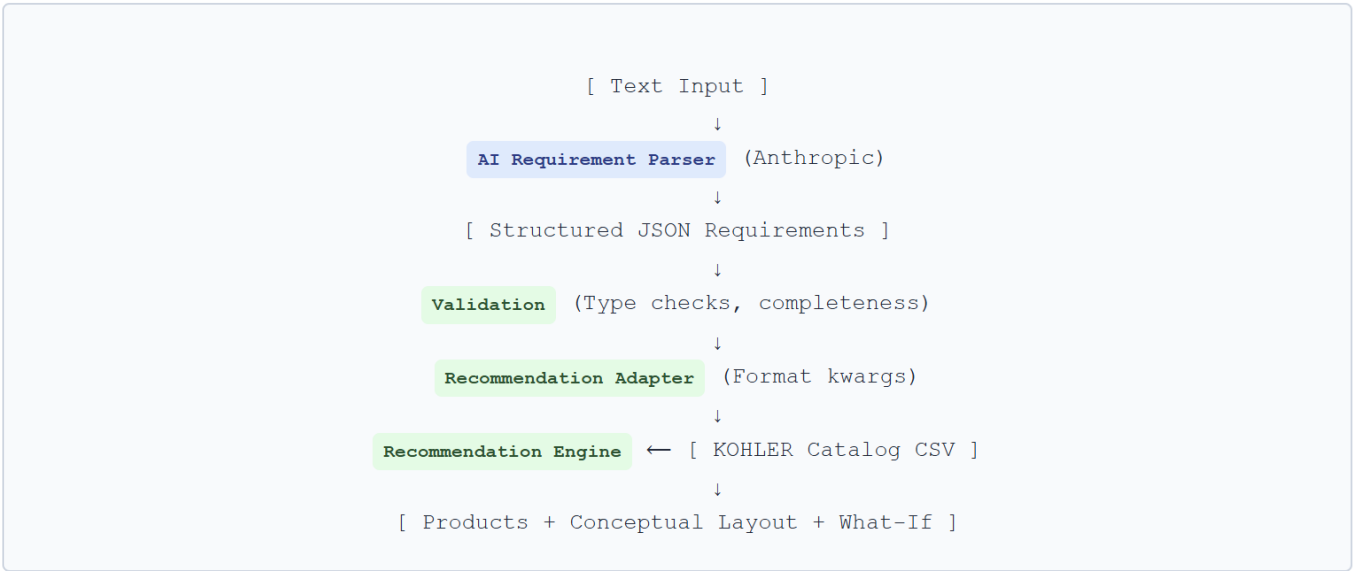
SECTION 10 — PROMPT-TO-FEATURE MAPPING

Prompt / Instruction	Location	Purpose	Output Used By
SYSTEM_PROMPT	llm_parser.py	Extract structured bathroom requirements from free-text descriptions while enforcing constraint rules.	_extract_json_from_response_text → validation.py

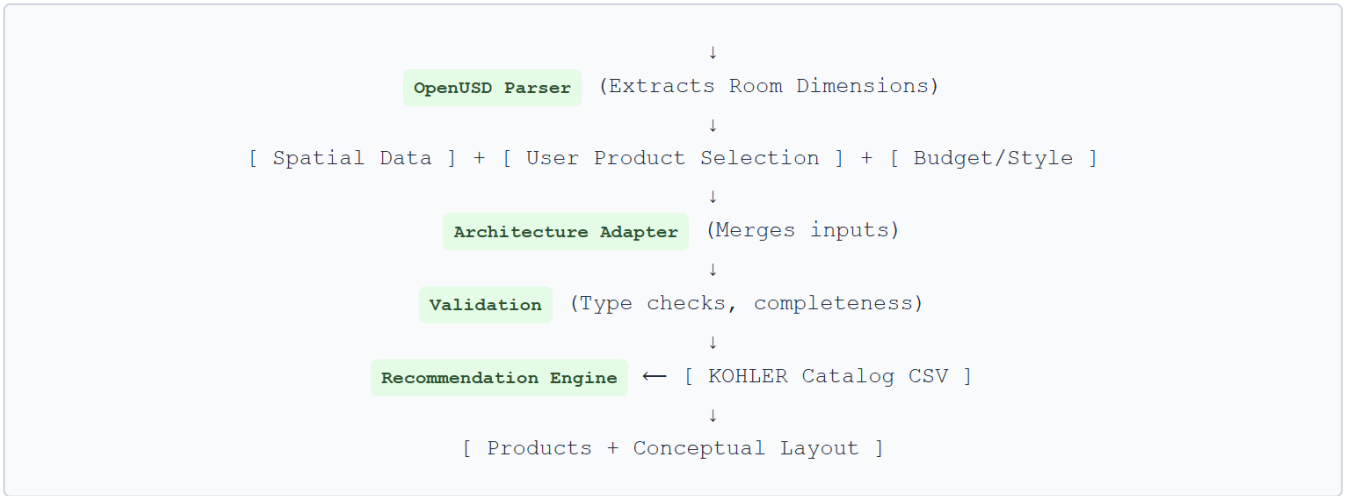
Note: Only one main system prompt is utilized in the codebase. The What-If feature re-uses this same prompt structure for delta extraction.

SECTION 11 — END-TO-END WORKFLOWS

WORKFLOW A: Natural Language (AI-Assisted)



WORKFLOW B: Spatial Architecture (Deterministic)[USD/USDZ File]



SECTION 12 — TECHNOLOGY SUMMARY

The following technologies and libraries are explicitly utilized within the provided source code:

- **Python 3** (Core logic and engines)
- **Streamlit** (Web UI framework inside `app.py`)
- **OpenUSD / pxr** (3D spatial file parsing)
- **Pandas** (Dataframe management for the `products.csv` catalog)
- **Anthropic SDK** (LLM integration)
- **python-dotenv** (Environment variable management)

SECTION 13 — LIMITATIONS

The current implementation includes the following verified limitations:

- **Demo Mode Simplicity:** The regex-based demo parser lacks semantic nuance. It relies on hard-coded keywords and patterns, which may fail on complex or non-standard phrasing.
- **USD Intent Separation:** The USD parser strictly reads physical geometries. It does not infer user purchase intent from existing objects.
- **Conceptual Visualization:** The SVG layout generator provides a visual planning aid (a topological arrangement), but is explicitly noted as "not architectural scale".
- **Missing Dimensions:** Products missing spatial footprints in the CSV (e.g., faucets) are ignored during spatial scoring to prevent false-negative rejections.

SECTION 14 — REPRODUCIBILITY

To run and review the project locally, follow these steps:

- 1 Install dependencies via `pip install -r requirements.txt`.
- 2 Configure the environment by creating a `.env` file in the root directory.
 - For Live AI mode: Set `LLM_API_KEY=YOUR_ANTHROPIC_KEY`
 - For Local Demo mode: Set `DEMO_MODE=true`
- 3 Run the web application using: `streamlit run app.py`

Prompt & Workflow Summary

The **KOHLER AI Bathroom Designer** securely bounds its LLM usage exclusively to intent extraction. Whether processing natural language via the Anthropic API or parsing physical space via OpenUSD, all data converges into a strict JSON contract. This contract undergoes rigorous deterministic validation before entering the Pandas-powered recommendation engine, guaranteeing that final KOHLER product layouts are always physically viable, mathematically sound, and hallucination-free

